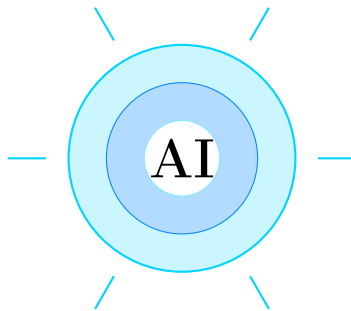


J.A.R.V.I.S.

Just A Rather Very Intelligent System

Complete System Documentation

Version 1.0.0



J.A.R.V.I.S. Development Team

May 27, 2026

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Key Features	5
1.3	Technology Stack	7
2	System Architecture	8
2.1	High-Level Architecture	8
2.2	Service Communication Architecture	9
2.3	Deployment Architecture	10
3	Core Components	12
3.1	Orchestrator Service	12
3.1.1	API Endpoints	12
3.1.2	WebSocket Protocol	12
3.1.3	Session Management	13
3.2	Agent Engine	13
3.2.1	The ReAct Loop	13
3.2.2	ToolRegistry	14
3.2.3	Tool Categories	15
3.2.4	Intent Classification	17
3.3	Memory Palace	17
3.3.1	Memory Types	17
3.3.2	Memory Importance Levels	18
3.3.3	The 20 Memory Palace Rooms	18
3.3.4	Memory Flow Diagram	19
3.4	LLM Client	19
3.4.1	Provider Architecture	19
3.4.2	Auto-Failover Priority Chain	19
4	Perception Systems	21
4.1	Vision System	21
4.1.1	Architecture	21
4.1.2	Data Flow	21
4.1.3	Detection Results	22
4.2	Hearing System	22
4.2.1	Architecture	22
4.2.2	Wake Word Processing Flow	23
4.3	Speech System	23

4.3.1	Architecture	23
4.3.2	Emotion Mapping	24
4.3.3	TTS Synthesis Pipeline	24
4.4	Action Service	25
5	Frontend Systems	26
5.1	React Application	26
5.1.1	Component Architecture	26
5.1.2	WebSocket Integration	26
5.1.3	State Management	27
5.2	3D Circular Equalizer	27
5.2.1	Architecture	27
5.2.2	State Machine	27
5.2.3	EQ Profiles per Mode	27
5.2.4	Visual Components	28
5.2.5	Signal Processing Algorithms	28
6	Configuration and Deployment	30
6.1	Configuration System	30
6.1.1	Environment Variables	30
6.2	Docker Deployment	31
6.2.1	Service Definitions	31
6.2.2	Dockerfile Structure	31
7	Data Models	32
7.1	Orchestrator Models	32
7.2	Memory Models	34
7.3	Speech Models	35
8	Extended Feature Modules	36
8.1	Color and Emotion Detection	36
8.2	Shopping Assistant	36
8.3	Email and Calendar Agent	37
8.4	Data, Media, and Advanced Tool Libraries	38
8.4.1	Data Tools (35+ functions)	39
8.4.2	Media Tools (30+ functions)	39
8.4.3	Advanced Tools (45+ functions)	39
8.5	Understand-Anything Integration	39
9	Roadmap	41
9.1	Feature Categories	41
9.1.1	High Priority (Implement First)	41
9.1.2	Medium Priority (Implement Next)	41
9.1.3	Low Priority (Nice to Have)	42
10	Appendix	43
10.1	Complete Tool List	43
10.2	Requirements	44
10.3	File Tree	45

List of Figures

2.1	High-Level System Architecture	9
2.2	Redis Pub/Sub Communication Pattern	10
2.3	Docker Compose Deployment Architecture	11
3.1	Agent ReAct Processing Loop	14
3.2	Memory Storage Flow	19
3.3	LLM Provider Hierarchy	19
4.1	Vision Data Flow	21
4.2	Wake Word Detection Flow	23
4.3	TTS Synthesis Pipeline with Fallback Chain	24
5.1	React Component Hierarchy	26
5.2	Equalizer State Machine	27
8.1	Color and Emotion Detection Pipeline	36
8.2	Email and Calendar Agent Architecture	37
8.3	Understand-Anything Multi-Agent Pipeline	40

List of Tables

1.1	Technology Stack	7
3.2	WebSocket Message Types	13
3.3	Session Data Fields	13
3.4	Tool Dataclass Fields	15
3.5	All Tool Categories	16
3.6	Intent Classification Patterns	17
3.7	Memory Types	17
3.8	Memory Importance Levels	18
3.9	Memory Palace Rooms	18
4.1	DetectionResult Fields	22
4.2	Emotion-to-Speech Parameters	24
4.3	Action Modules and Their Capabilities	25
5.2	EQ Profiles per Mode	28
6.2	Docker Compose Services	31
7.1	Core Pydantic Models	33
7.2	Memory Data Models	34
7.3	TTS Request/Response Models	35
8.1	Shopping Assistant Tools	37
10.2	Python Dependencies by Category	45

Chapter 1

Introduction

1.1 Project Overview

J.A.R.V.I.S. (Just A Rather Very Intelligent System) is a modular, production-ready AI personal assistant inspired by the fictional J.A.R.V.I.S. from the Marvel Cinematic Universe. It is designed to operate as a comprehensive digital companion with capabilities spanning vision, hearing, speech, long-term memory, smart home control, file management, web interaction, and much more.

The system is built using a **microservice architecture** with Python-based backend services communicating via Redis Pub/Sub, a React + Three.js frontend for the 3D avatar interface, and a standalone HTML5 equalizer visualization. It supports multiple LLM providers with automatic failover, maintains a Memory Palace with 20 themed rooms using ChromaDB vector storage, and exposes **300+ tools** across 40+ categories through a LangChain-inspired ReAct (Reasoning + Acting) agent loop with multi-agent orchestration capabilities.

1.2 Key Features

- **Multi-LLM Support:** OpenAI GPT-4, Anthropic Claude, DeepSeek, OpenRouter, local models (Ollama/llama.cpp) with automatic provider failover
- **Memory Palace:** 20 themed rooms with episodic, semantic, procedural, and spatial memory types stored in ChromaDB
- **300+ Tools:** 40+ categories spanning filesystem, web search, email, calendar, smart home, finance, news, sports, entertainment, code execution, database, shopping, color/emotion detection, face recognition, data manipulation, media processing, text intelligence, security, and more
- **Vision System:** YOLOv8 object detection, face recognition, QR/barcode reading, color detection with named-color mapping, palette generation, real-time emotion detection from face images
- **Hearing System:** Always-on microphone with WebRTC VAD, OpenAI Whisper transcription, wake word detection
- **Speech System:** Multi-engine TTS (ElevenLabs, Coqui, gTTS), emotional speech with 8 emotions, viseme/lip-sync generation

- **3D Equalizer:** Three.js circular audio visualizer with 72 radial bars, 5 EQ bands, beat detection, ADSR envelope, bloom post-processing
- **React Frontend:** Three.js 3D avatar, real-time chat, camera feed, audio visualizer, emotion controls
- **Shopping Assistant:** Product search, category browsing, price comparison, policy search, order tracking with SQLite + FAISS vector store
- **Email Agent:** Full email management (search, send, draft, reply, forward, archive, trash), calendar management with HITL (Human-in-the-Loop) approval pattern
- **Understand-Anything Integration:** Interactive knowledge graph generation for any codebase, multi-agent pipeline with tree-sitter + LLM hybrid analysis, guided codebase tours, diff impact analysis
- **Docker Deployment:** 8-container microservice stack with persistent volumes and health checks

1.3 Technology Stack

Layer	Technologies
Backend Framework	Python 3.12, FastAPI, Uvicorn
Agent Framework	LangChain-inspired ReAct loop with custom ToolRegistry, multi-agent orchestration, subagent spawning
LLM Providers	OpenAI GPT-4, Anthropic Claude, DeepSeek, OpenRouter, Ollama, llama.cpp
Vector Database	ChromaDB with sentence-transformers embeddings, FAISS for product/policy search
Message Broker	Redis 7 (async Pub/Sub with auto-reconnect)
Object Detection	YOLOv8 nano, OpenCV Haar Cascade
Face Recognition	face_recognition library, OpenCV, DeepFace (emotion, age, gender, race)
Color Detection	K-means clustering, named-color mapping (24+ colors), KNN, Color Thief algorithm
Speech-to-Text	OpenAI Whisper (local tiny model or API)
Voice Activity Detection	WebRTC VAD
Text-to-Speech	ElevenLabs API, Coqui TTS, gTTS
Frontend	React 18, Three.js 0.150.0, react-three-fiber
Equalizer	Three.js (standalone HTML with importmap), EffectComposer, UnrealBloomPass
Database	SQLite, PostgreSQL, MySQL (via action tools); SQLite for product/email stores
Smart Home	MQTT, Home Assistant integration
Semantic Routing	TF-IDF vectorizer for query classification (shopping vs. general)
Knowledge Graphs	Tree-sitter + LLM hybrid, multi-agent pipeline, interactive dashboard
Containerization	Docker, Docker Compose
Testing	pytest, pytest-asyncio, httpx, pytest-cov

Table 1.1: Technology Stack

Chapter 2

System Architecture

2.1 High-Level Architecture

The system follows a microservice architecture pattern where each major capability is isolated in its own service, all connected through Redis Pub/Sub for asynchronous communication. The Orchestrator acts as the central brain, routing requests between services and managing the agent's reasoning loop.

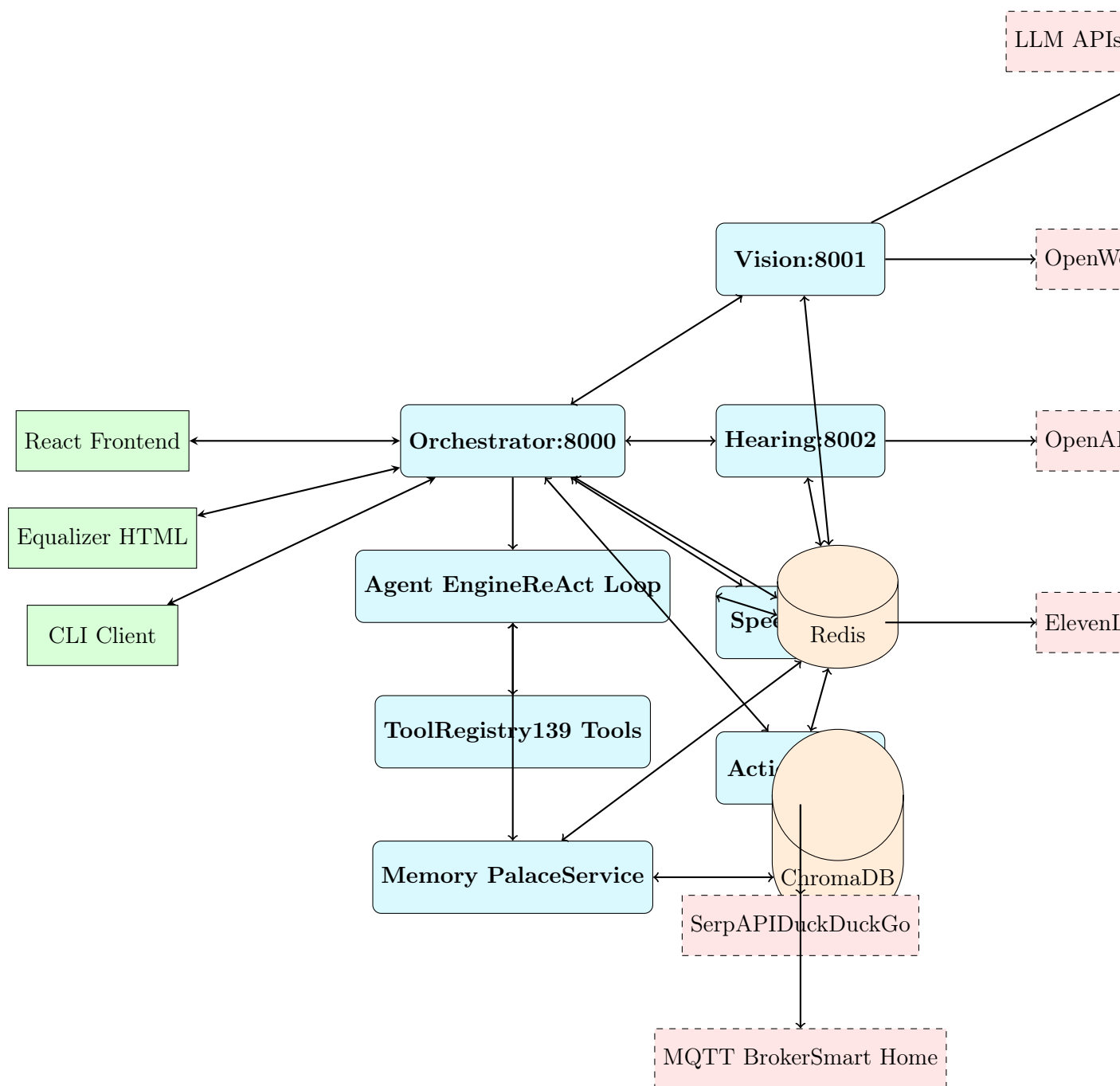


Figure 2.1: High-Level System Architecture

2.2 Service Communication Architecture

All services communicate through Redis Pub/Sub using a request/response pattern. The `RedisPubSub` class in `backend/shared/redis_client.py` provides automatic reconnection, channel-based message routing, and typed RPC stubs defined in `grpc_stubs.py`.

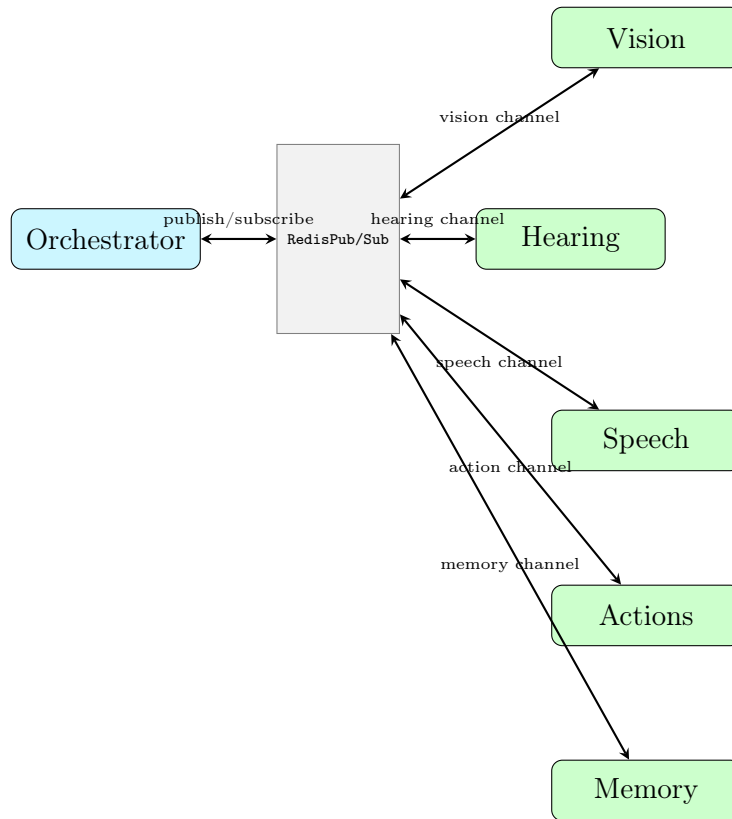


Figure 2.2: Redis Pub/Sub Communication Pattern

2.3 Deployment Architecture

The system is containerized using Docker Compose with 8 services, persistent volumes for Redis and ChromaDB, and a shared bridge network.

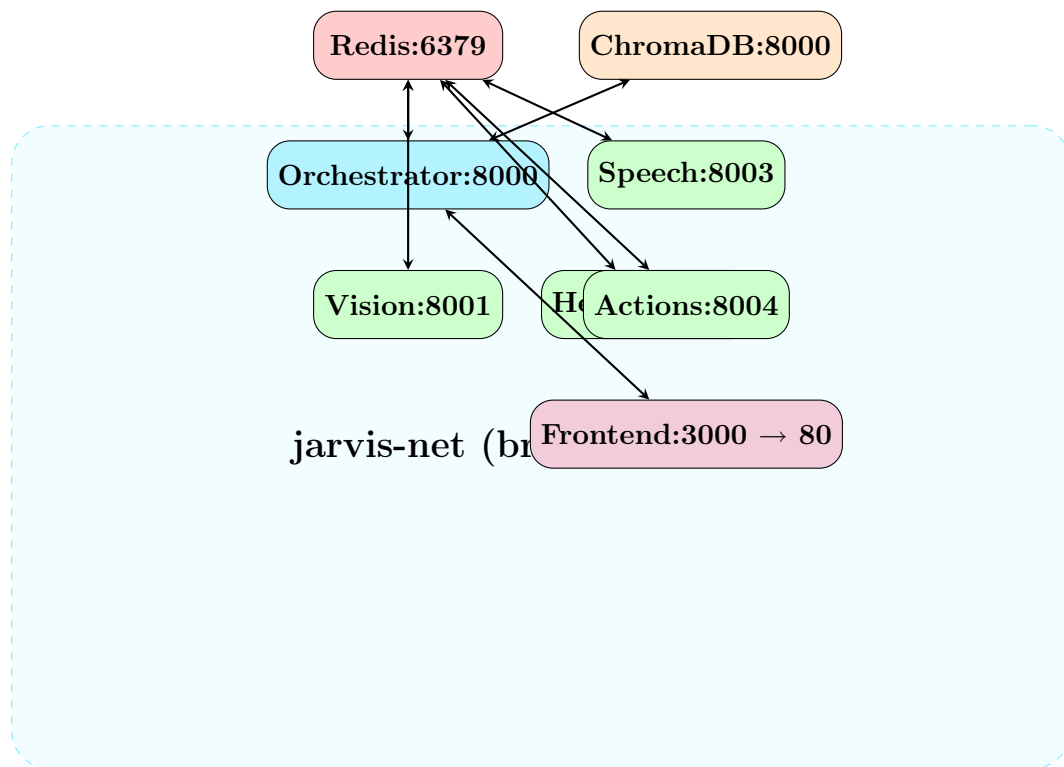


Figure 2.3: Docker Compose Deployment Architecture

Chapter 3

Core Components

3.1 Orchestrator Service

The Orchestrator is the central brain of J.A.R.V.I.S., implemented as a FastAPI application in `backend/orchestrator/main.py`. It manages WebSocket connections, routes chat requests to the agent, coordinates between microservices, and handles session management.

3.1.1 API Endpoints

Method	Endpoint	Description	Response Model
GET	/health	Health check with uptime, session count, memory status	StatusResponse
POST	/chat	Main conversation endpoint	ChatResponse
POST	/chat/stream	SSE streaming chat with token-by-token output	Event Stream
GET	/avatar/expression	Get avatar blend shapes for emotion	ExpressionResponse
POST	/vision/scene	Scene understanding from camera image	SceneResponse
POST	/session	Create a new session	SessionInfo
GET	/session/{session_id}	Get session information	SessionInfo
GET	/status	System status with tool listing	StatusResponse
WS	/ws	Bidirectional WebSocket (chat, audio, image, commands)	WebSocketMessage

3.1.2 WebSocket Protocol

The WebSocket at `/ws` supports multiple message types for bidirectional communication:

Message Type	Payload
chat	{ text: string, session_id: string, emotion?: string }
audio	{ audio: base64-string, session_id: string }
image	{ image: base64-string, session_id: string }
command	{ action: string, params?: object }
mode	{ mode: "listening" "thinking" "speaking" }
emotion	{ emotion: string }
visemes	{ frames: array }
ping/pong	{ }

Table 3.2: WebSocket Message Types

3.1.3 Session Management

Sessions are stored in-memory as a dictionary mapping session IDs to session objects:

2gray!10white

Field	Description
created_at	Session creation timestamp
last_active	Timestamp of last interaction
message_count	Total messages in session
emotion	Current emotional state
user_name	Identified user name

Table 3.3: Session Data Fields

3.2 Agent Engine

3.2.1 The ReAct Loop

The agent uses a Reasoning + Acting (ReAct) loop pattern. Each iteration follows: **Thought** → **Action** → **Observation** → **Response**.

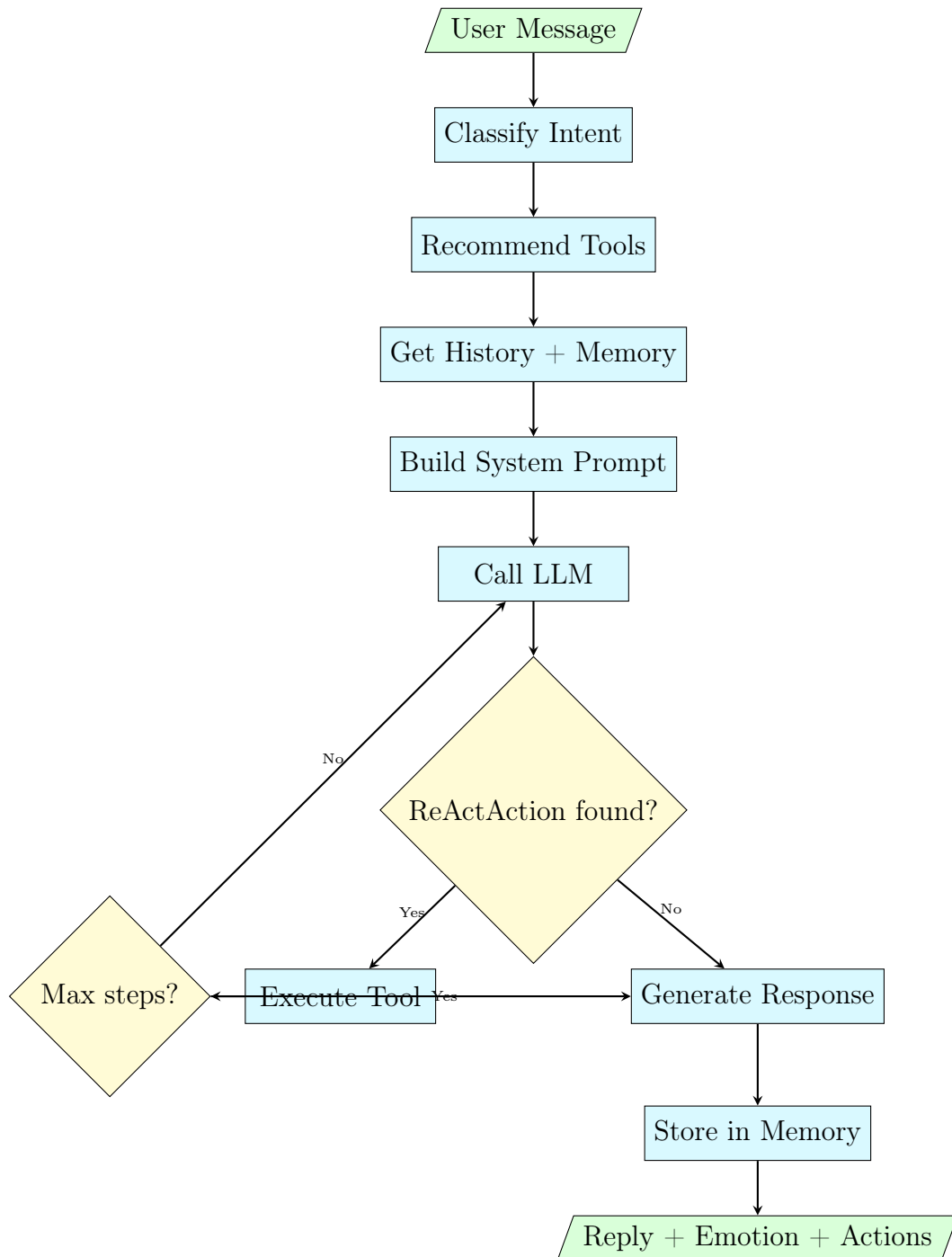


Figure 3.1: Agent ReAct Processing Loop

3.2.2 ToolRegistry

The `ToolRegistry` class (in `backend/orchestrator/agent.py`) manages all 300+ tools. Each tool is registered as a `Tool` dataclass with the following fields:

2gray!10white

Field	Description
name	Unique tool identifier
description	Natural-language description for LLM selection
parameters	JSON Schema describing accepted parameters
handler	Async callable that executes the tool
requires_confirmation	Whether HITL confirmation is needed
category	Logical category grouping (vision, data, finance, etc.)

Table 3.4: Tool Dataclass Fields

3.2.3 Tool Categories

The 300+ tools are organized into the following categories:

2gray!10white		
Category	Count	Example Tools
utility	15	get_time, get_world_time, random_number, random_choice, color_convert, math_calculate, query_yesno
information	5	search_web, get_weather, search_academic, scrape_page, firecrawl_scrape
communication	5	send_email, read_inbox, search_emails, slack_send, discord_send
productivity	8	create_calendar_event, set_reminder, get_upcoming_events, schedule_reminder, schedule_at, list_scheduled_tasks, cancel_task, notion_create
smart_home	6	control_light, set_temperature, set_light_scene, set_humidity, lock_door, get_energy_usage
system	15+	mouse_move, mouse_click, keyboard_type, keyboard_hotkey, window_manage, screenshot, system_settings_control, multi_monitor, startup_program, macro_record, macro_run, orchestrate_sequence
memory	5	remember, recall, summarize_memory, remember_topic, remember_preference
vision	12+	detect_colors, analyze_scene, extract_dominant_colors, color_name_from_rgb/hex, color_palette_generate, color_analyze_image_url/local, detect_emotion_face, detect_faces_image, analyze_face_attributes, compare_faces
avatar	1	get_avatar_expression
filesystem	20+	read_file, write_file, list_files, delete_file, search_files, file_hash, file_diff, file_compress, file_type_detect, obsidian_* (5 tools)
data	35+	read_csv, query_csv, read_excel, read_json, query_json, str_reverse, str_upper, str_split, str_join, list_shuffle, list_unique, list_flatten, dict_merge, json_flatten, stat_mean, stat_median, stat_correlation
development	10+	run_python, run_javascript, git_* (6 tools), generate_code, generate_git_commit, generate_dockerfile
finance	10+	get_stock_price, get_market_indices, get_company_info, get_stock_historical, get_top_movers, convert_currency, get_exchange_rates, polygon_stock_price, stripe_*
news	4	get_news, get_stock_news, scrape_hacker_news, scrape_news
crypto	2	get_crypto_price, get_trending_crypto
sports	1	get_sports_scores
entertainment	6+	get_movie_info, get_trending_movies, spotify_*, recommend_movie, get_music_recommendations, get_daily_quote
food	2	get_nutrition, search_recipes
reference	6+	define_word, get_synonyms, get_holidays, get_ip_info, search_books, scrape_wikipedia
fun	4	get_joke, get_quote, get_random_fact, desk-

3.2.4 Intent Classification

The agent classifies user messages into 21 intent patterns using keyword matching in `_classify_intent()`:

Intent	Keywords/Patterns
greeting	hello, hi, hey, good morning, good evening, yo
farewell	goodbye, bye, see you, good night, talk later
time_query	time, date, what day, current time, what's the date
weather_query	weather, temperature, forecast, rain, sunny
search_query	search, find, look up, google, research
knowledge_query	what is, who is, explain, define, tell me about, how does
email_task	email, send mail, compose, draft, write to
calendar_task	calendar, event, schedule, appointment, meeting
reminder_task	remind, reminder, remember to, notify me
file_task	file, read, write, save, open, create, delete, list, folder, directory
music_task	play, music, song, spotify, playlist
code_task	code, program, script, function, algorithm, debug
joke_request	joke, funny, humor, make me laugh
quote_request	quote, inspiration, motivate
stock_query	stock, share price, market, ticker, aapl, tsla
chart_request	chart, graph, plot, visualize
help_request	help, what can you do, capabilities, commands
status_query	status, how are you, what's up, are you there
memory_query	remember, recall, what do you know about, memories
translation	translate, in spanish, in french, how do you say
debate_request	debate, argue, discuss, pros and cons, compare viewpoints
general_conversation	(default – no specific pattern matched)

Table 3.6: Intent Classification Patterns

3.3 Memory Palace

3.3.1 Memory Types

The Memory Palace system (`backend/memory/service.py`) implements a sophisticated long-term memory architecture with four distinct memory types:

Memory Type	Description
Episodic	Personal experiences and events (e.g., "User asked about weather yesterday at 3pm")
Semantic	Facts and general knowledge (e.g., "User's favorite color is blue")
Procedural	How-to knowledge and processes (e.g., "How the user likes their emails formatted")
Spatial	Spatial and location-based memories (e.g., "Smart light in living room is called 'main'")

Table 3.7: Memory Types

3.3.2 Memory Importance Levels

Each memory trace has an importance level that affects its retention and retrieval priority:

Level	Description
TRIVIAL	Low-value, easily forgettable (e.g., "User said 'ok'")
LOW	Minor facts or casual mentions
NORMAL	Standard memories (conversation summaries)
HIGH	Important information (user preferences, key facts)
CRITICAL	Essential data (passwords, configurations, critical instructions)

Table 3.8: Memory Importance Levels

3.3.3 The 20 Memory Palace Rooms

The memory palace consists of 20 themed rooms, each designed for a specific domain of knowledge:

Room Name	Description / Topic
Grand Foyer	First impressions, user identity, core preferences
Library	Books, learning, knowledge, academic interests
Workshop	Projects, development, code, technical skills
Garden	Nature, hobbies, relaxation, personal growth
Kitchen	Food, recipes, nutrition, cooking preferences
Living Room	Social interactions, family, friends, relationships
Study Room	Work, career, productivity, professional goals
Music Room	Music, playlists, instruments, audio preferences
Art Gallery	Visual arts, design, creativity, aesthetic preferences
Home Theater	Movies, TV shows, entertainment preferences
Gym	Fitness, health, exercise routines, wellness goals
Travel Desk	Travel plans, destinations, trips, geography
Command Center	Smart home configuration, devices, automation rules
Vault	Security, passwords, sensitive information (encrypted)
Observatory	Science, space, technology, research interests
Greenhouse	Environment, sustainability, climate preferences
Archive	Historical data, past conversations, long-term trends
Workshop II	Side projects, experiments, creative endeavors
Gaming Room	Games, sports, competitive interests
Zen Garden	Meditation, mindfulness, emotional state, mood patterns

Table 3.9: Memory Palace Rooms

3.3.4 Memory Flow Diagram

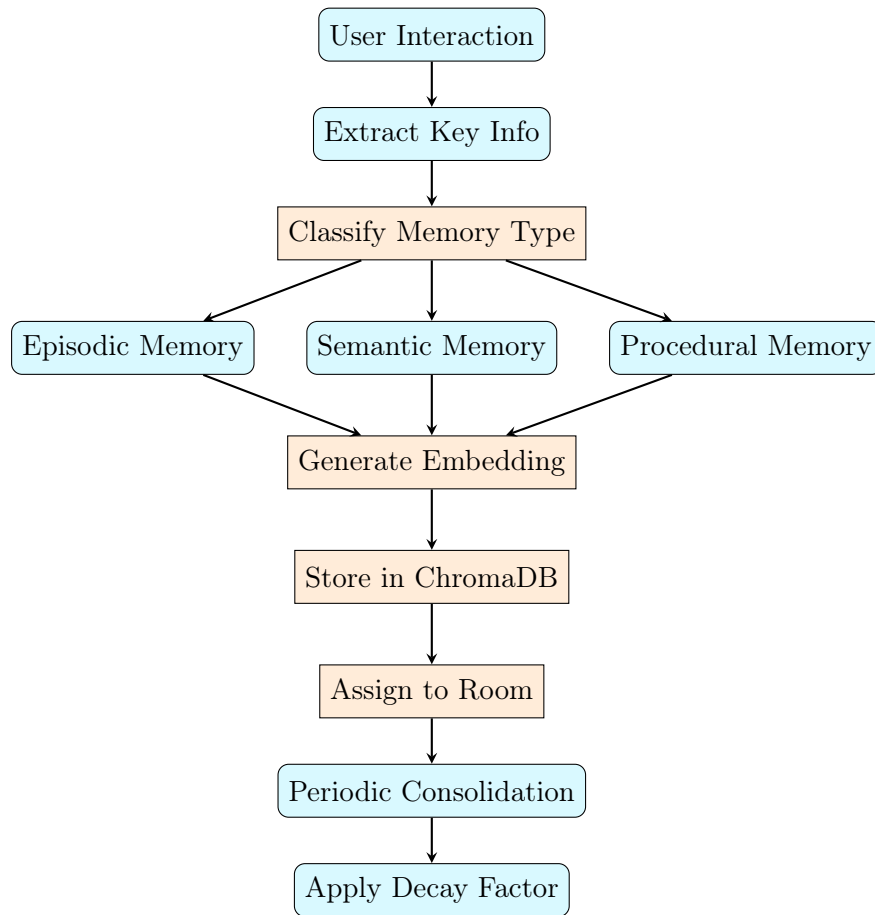


Figure 3.2: Memory Storage Flow

3.4 LLM Client

3.4.1 Provider Architecture

The `LLMClient` class (`backend/shared/llm_client.py`) provides a unified interface to multiple LLM providers with automatic failover:

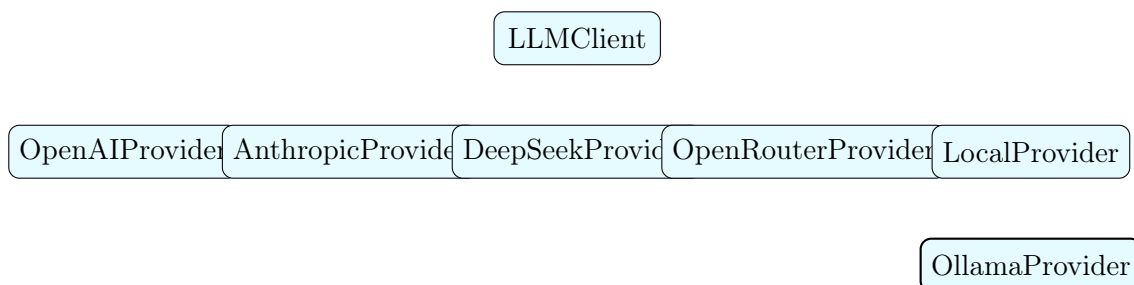


Figure 3.3: LLM Provider Hierarchy

3.4.2 Auto-Failover Priority Chain

The LLM client builds a prioritized provider chain based on configured API keys:

1. **OpenAI** – if `OPENAI_API_KEY` is set (models: gpt-4, gpt-4-turbo, gpt-3.5-turbo)
2. **Anthropic** – if `ANTHROPIC_API_KEY` is set (models: claude-3-opus, claude-3-sonnet, claude-3-haiku)
3. **DeepSeek** – if `DEEPSEEK_API_KEY` is set (uses OpenAI-compatible API)
4. **OpenRouter** – if `OPENROUTER_API_KEY` is set (gateway to 200+ models)
5. **Local** – if `LOCAL_LLM_URL` is set (llama.cpp, vLLM, etc.)
6. **Ollama** – if available at `http://localhost:11434`

If one provider fails (rate limit, timeout, API error), the client automatically falls through to the next available provider in the chain.

Chapter 4

Perception Systems

4.1 Vision System

4.1.1 Architecture

The Vision service (`backend/vision/`) provides real-time computer vision capabilities:

- **Object Detection:** YOLOv8 nano for real-time object detection with configurable confidence threshold
- **Face Recognition:** `face_recognition` library with known-face database
- **QR/Barcode Detection:** `pyzbar` library
- **Color Detection:** K-means clustering for dominant color extraction
- **Frame Capture:** Support for webcam, RTSP streams, and screen capture

4.1.2 Data Flow

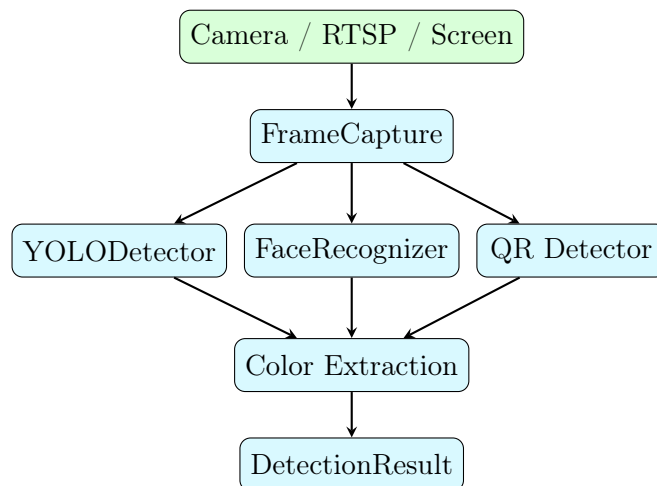


Figure 4.1: Vision Data Flow

4.1.3 Detection Results

2gray!10white

Field	Description
objects	List of DetectedObject (label, confidence, bounding box, dominant color)
labels	All detected class labels
detection_time_ms	Inference time in milliseconds
frame_width	Frame width in pixels
frame_height	Frame height in pixels
dominant_colors	RGB tuples from K-means clustering

Table 4.1: DetectionResult Fields

4.2 Hearing System

4.2.1 Architecture

The Hearing service (`backend/hearing/`) handles audio capture and speech recognition:

- **Microphone Capture:** PyAudio with background capture thread
- **Voice Activity Detection:** WebRTC VAD with configurable threshold and silence duration
- **Speech-to-Text:** OpenAI Whisper (local tiny model for speed, API for accuracy)
- **Wake Word:** "Jarvis" and "Hey Jarvis" detection with configurable sensitivity
- **Audio Ring Buffer:** Stores recent audio for context-aware processing

4.2.2 Wake Word Processing Flow

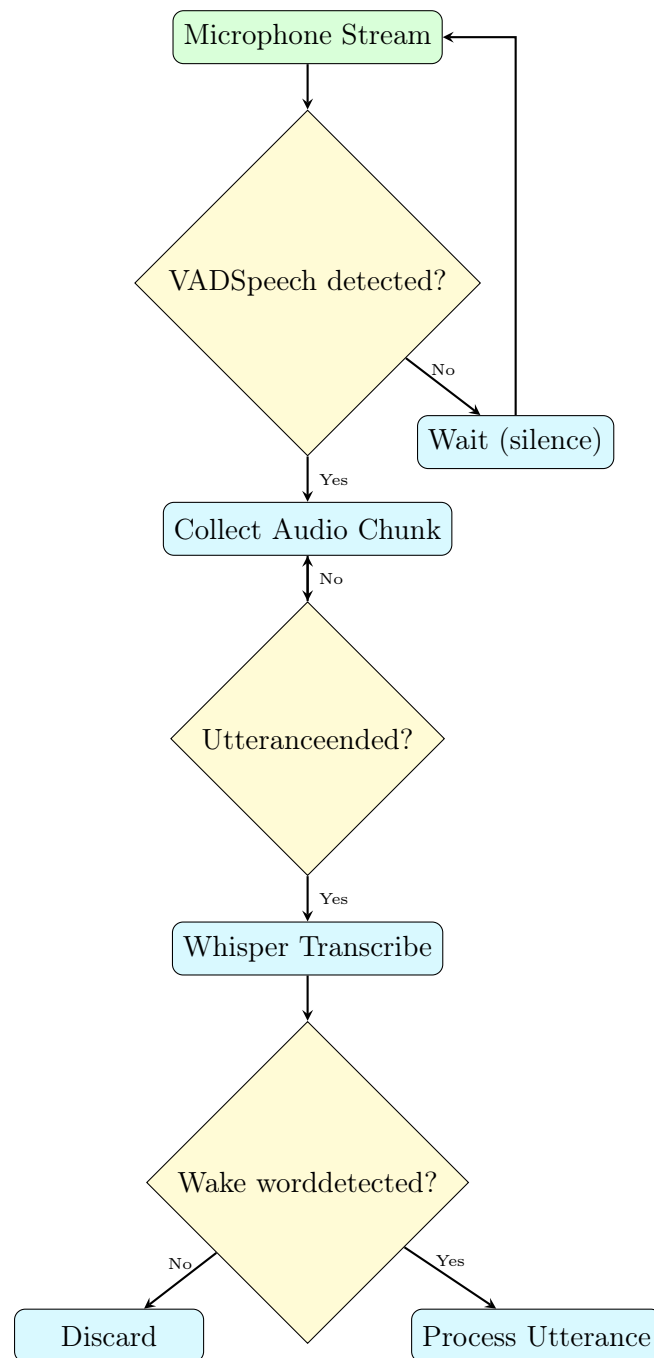


Figure 4.2: Wake Word Detection Flow

4.3 Speech System

4.3.1 Architecture

The Speech service (`backend/speech/`) provides text-to-speech synthesis with emotional expression:

- **Primary Engine:** ElevenLabs API (highest quality, emotional range)

- **Fallback 1:** Coqui TTS (local open-source TTS)
- **Fallback 2:** gTTS (Google Text-to-Speech, last resort)
- **Emotional Support:** 8 emotions mapped to rate/pitch adjustments
- **Lip Sync:** Viseme generation for avatar mouth animation
- **Audio Effects:** Radio, echo, robotic, whisper, slow, fast
- **Caching:** LRU cache to avoid re-synthesizing common phrases

4.3.2 Emotion Mapping

Emotion	Effect	Rate Multiplier	Pitch Shift
neutral	Standard speech	1.0	0
happy	Brighter, slightly faster	1.15	+1.5
excited	Fast, high energy	1.25	+2.0
serious	Slower, deeper	0.85	-2.0
calm	Slow, soothing	0.75	-1.0
angry	Faster, aggressive	1.1	-1.5
sad	Slow, quiet, monotone	0.7	-2.5
whisper	Quiet, breathy	0.9	-1.0

Table 4.2: Emotion-to-Speech Parameters

4.3.3 TTS Synthesis Pipeline

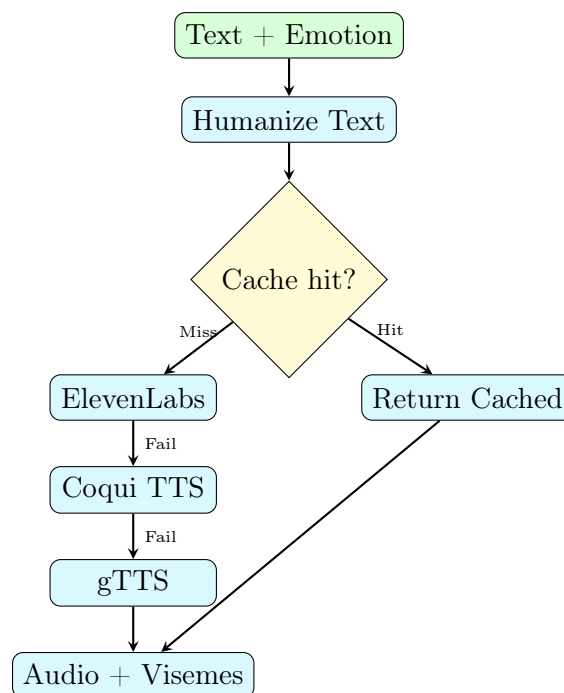


Figure 4.3: TTS Synthesis Pipeline with Fallback Chain

4.4 Action Service

The Actions service (`backend/actions/`) provides executable capabilities:

Module	Capabilities
<code>api_integrations.py</code>	16 API integrations: stocks, news, crypto, forex, sports, movies, music, nutrition, dictionary, jokes, quotes, holidays, IP geo, NASA, books, world time
<code>file_handler.py</code>	Read/write CSV, Excel, PDF, JSON, YAML, XML, DOCX, PPTX, Markdown, SVG generation, LaTeX compilation
<code>web_scraper.py</code>	Static HTML scraping, Playwright JS rendering, robots.txt compliance, article/product/table extraction
<code>web_search.py</code>	SerpAPI, DuckDuckGo, OpenWeatherMap
<code>email.py</code>	SMTP email sending with HTML support
<code>email_reader.py</code>	IMAP inbox reading with search
<code>calendar.py</code>	Google Calendar event creation
<code>code_sandbox.py</code>	Isolated Python and JavaScript execution with timeout
<code>database.py</code>	SQLite, PostgreSQL, MySQL query execution
<code>home_assistant.py</code>	MQTT smart home control (lights, temperature)
<code>image_gen.py</code>	DALL-E 3 image generation
<code>music.py</code>	Spotify search, play, pause, current track
<code>visualizations.py</code>	Matplotlib charts: line, bar, pie, scatter, histogram, heatmap, 3D surface, animated charts
<code>system_commands.py</code>	Open applications, take screenshots, system info
<code>git_ops.py</code>	Git status, log, commit, push, pull, branch
<code>document_qa.py</code>	Document ingestion and RAG-based Q&A
<code>notifications.py</code>	Desktop and sound notifications
<code>scheduler.py</code>	In-memory task scheduling with reminders
<code>avatar.py</code>	Expression mapping, viseme shapes, head movement, body language

Table 4.3: Action Modules and Their Capabilities

Chapter 5

Frontend Systems

5.1 React Application

5.1.1 Component Architecture

The React frontend at `frontend/src/` is organized into six components:

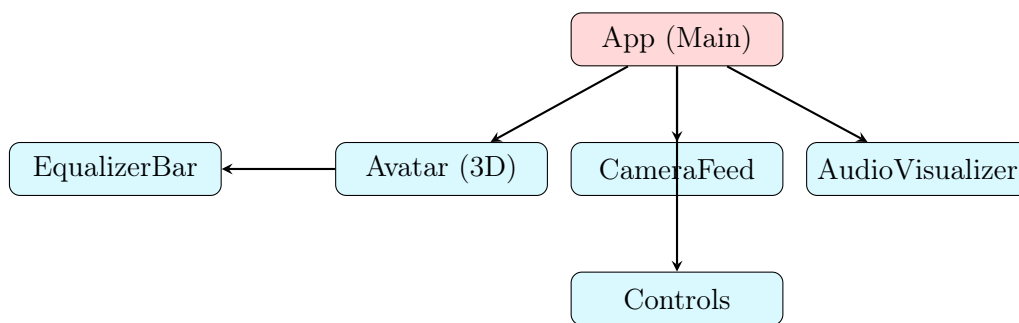


Figure 5.1: React Component Hierarchy

- **App.js:** Main application with WebSocket manager, audio capture, state management, and three-panel layout
- **Avatar.js:** Three.js 3D avatar using react-three-fiber with MeshDistortMaterial, equalizer bars, and emotion-driven animation
- **CameraFeed.js:** getUserMedia-based camera capture with snapshot capability
- **AudioVisualizer.js:** Real-time 16-bar equalizer visualization
- **Controls.js:** Emotion override panel and command buttons

5.1.2 WebSocket Integration

The frontend maintains a persistent WebSocket connection to the orchestrator with auto-reconnect (3-second delay). Messages flow bidirectionally:

- **Frontend** → **Backend:** Chat messages, audio chunks (base64), camera images (base64), commands
- **Backend** → **Frontend:** Chat replies, emotion updates, viseme frames, audio levels, listening status, mode changes

5.1.3 State Management

The App component maintains these state variables:

State	Type	Description
sessionId	string	Auto-generated session identifier
ws	WebSocket	Active WebSocket connection
messages	array	Chat message history {role, text}
inputText	string	Current text input value
isConnected	boolean	WebSocket connection status
emotion	string	Current emotion (neutral, happy, excited, etc.)
isSpeaking	boolean	J.A.R.V.I.S. is speaking
isListening	boolean	J.A.R.V.I.S. is listening
audioLevel	number	Current audio level (0-1)
visemes	array	Lip-sync frame data
equalizerData	array	16-bin frequency data
micAvailable	boolean	Microphone accessibility

5.2 3D Circular Equalizer

5.2.1 Architecture

The standalone `equalizer.html` is a Three.js-powered 3D audio visualizer with 72 radial bars, beat detection, ADSR envelope, motion blur, particle systems, and WebSocket integration for automatic mode switching.

5.2.2 State Machine

The equalizer has four states driven by backend WebSocket mode events:

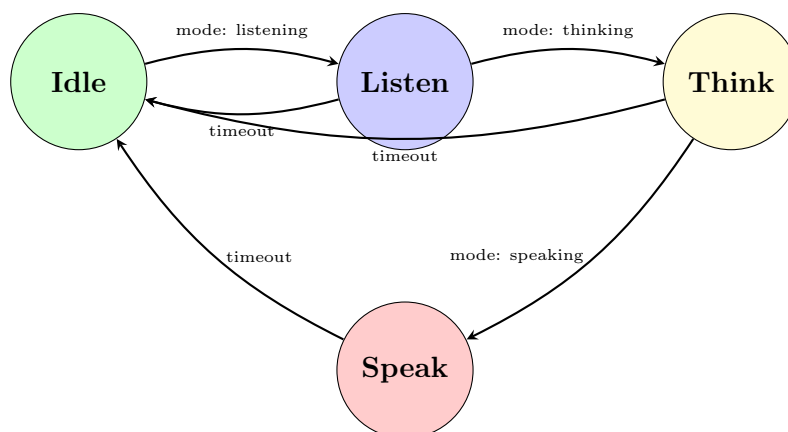


Figure 5.2: Equalizer State Machine

5.2.3 EQ Profiles per Mode

Each state has a unique EQ profile with gain offsets per frequency band, hue, and saturation:

State	Label	Low Bass	Bass	Mids	Presence	Treble	Hue
Idle	CALM / STANDBY	+2 dB	+3 dB	0 dB	-1 dB	-2 dB	0.55
Listen	LISTENING	0 dB	0 dB	+2 dB	+3 dB	-1 dB	0.52
Think	THINKING	0 dB	0 dB	0 dB	0 dB	0 dB	0.60
Speak	SPEAKING	0 dB	+2 dB	+2 dB	+4 dB	0 dB	0.48

Table 5.2: EQ Profiles per Mode

5.2.4 Visual Components

Component	Description	Count
Radial Bars	72 bars arranged in circle with height driven by frequency + EQ gain + motion blur trails	72
Ambient Particles	Floating particles with color variation (blue, purple, pink)	2800
Trail Particles	Ghost particles for motion blur effect	1500
Particle Bursts	2D canvas overlay bursts on beat detection	200 max
Neon Rings	3 decorative torus rings (outer, mid, inner)	3
Spin Rings	2 rotating torus rings at different speeds	2
Core Orb	Central icosahedron sphere with wireframe overlay and floating torus	3 meshes
Orbit Dots	3 orbiting dots visible in THINK mode	3
Bass Rings	2 torus rings that pulse with bass energy	2
Spectral History	Circular waterfall visualization (inner ring overlay)	40 frames
Time-Domain Waveform	Circular waveform overlay (listen mode)	200 samples

5.2.5 Signal Processing Algorithms

The equalizer implements 8 signal processing algorithms:

1. **Log-Scale FFT Mapping:** Perceptual frequency distribution using quadratic mapping: $\text{bin}_i = \left(\frac{i}{N}\right)^2 \times \text{FFT_size}$
2. **Beat Detection:** Energy-based transient detector comparing instantaneous variance to historical variance with cooldown
3. **ADSR Envelope:** Attack $t_a = 0.08$, Decay $t_d = 0.20$, Sustain $s = 0.55$, Release $t_r = 0.35$

4. **Motion Blur Trails:** Ring buffer of 5 frames with exponential decay weighting ($w_t = e^{-t \times 0.6}$)
5. **EQ Gain Scaling:** $\text{scale}(db) = 1.0 + db \times 0.04$
6. **Transition Ramping:** easeInOutCubic smoothing over 200ms
7. **HSL Color Mapping:** Hue from EQ profile + normalized amplitude ($\times 0.12$), saturation from profile, lightness $0.55 + \text{norm} \times 0.3$
8. **Chromatic Aberration:** RGB emissive shift on core orb material with exponential decay ($\times 0.92$ per frame)

Chapter 6

Configuration and Deployment

6.1 Configuration System

6.1.1 Environment Variables

The configuration system (`backend/shared/config.py`) loads all settings from environment variables with sensible defaults:

Category	Variable
Core	USER_NAME JARVIS_WAKE_WORD LOG_LEVEL JARVIS_DATA_DIR
API Keys	OPENAI_API_KEY ANTHROPIC_API_KEY OPENROUTER_API_KEY ELEVENLABS_API_KEY
Redis	REDIS_HOST REDIS_PORT
ChromaDB	CHROMA_HOST CHROMA_PORT CHROMA_COLLECTION
LLM	LLM_PROVIDER LLM_MODEL
Memory	MEMORY_PALACE_ENABLED MEMORY_PALACE_ROOMS MEMORY_DIMENSION
Speech	TTS_ENGINE TTS_VOICE_ID TTS_CACHE_ENABLED
Hearing	WHISPER_MODEL VAD_THRESHOLD WAKE_WORD_SENSITIVITY
Services	ORCHESTRATOR_PORT / VISION_PORT / HEARING_PORT / SPEECH_PORT

6.2 Docker Deployment

6.2.1 Service Definitions

Service	Image	Port	Depends On
redis	redis:7-alpine	6379	—
chroma	chromadb/chroma:latest	8000	—
orchestrator	jarvis-backend	8000	redis, chroma
vision	jarvis-backend	8001	redis
hearing	jarvis-backend	8002	redis
speech	jarvis-backend	8003	redis
action	jarvis-backend	8004	redis
frontend	nginx:alpine	3000:80	orchestrator

Table 6.2: Docker Compose Services

6.2.2 Dockerfile Structure

The backend runs on Python 3.11-slim with system dependencies (build-essential, ffmpeg, libsndfile1, libportaudio2), exposes ports 8000–8004, and starts via Uvicorn on port 8000.

Chapter 7

Data Models

7.1 Orchestrator Models

All Pydantic models defined in `backend/orchestrator/models.py`:

2gray!10white

Model	Field	Description
ChatRequest	message	User's text message
	session_id	Session identifier (default: "default")
	emotion	Emotion hint (default: "neutral")
	image	Optional base64-encoded image
	context	Optional arbitrary context dictionary
ChatResponse	reply	Generated response text
	session_id	Session identifier
	emotion	Detected emotion
	audio_url	Optional TTS audio URL
	visemes	Optional lip-sync viseme sequence
	actions_taken	List of executed tool names
	memory_updated	Whether memory was modified
	processing_time_ms	Processing latency
	confidence	Response confidence score (0–1)
	intent	Classified intent
	tool_recommendations	Suggested follow-up tools
StatusResponse	status	Service health ("ok", "degraded", "error")
	service	Service name
	version	Version string
	uptime_seconds	Elapsed time since start
	active_sessions	Concurrent session count
	memory_status	Memory backend status
	services	Per-service health map
WebSocketMessage	type	Message type (chat, audio, video, command, emotion, viseme)
	payload	Arbitrary message payload
	session_id	Session identifier
	timestamp	Unix timestamp of message

Table 7.1: Core Pydantic Models

7.2 Memory Models

2gray!10white

Model	Field	Description
MemoryType (Enum)	EPISODIC	Experiential memories (conversations, events)
	SEMANTIC	Factual knowledge (preferences, facts)
	PROCEDURAL	Skill/habit knowledge (routines, patterns)
	SPATIAL	Location-based memories (rooms, places)
MemoryImportance (Enum)	TRIVIAL	Minor casual mentions
	LOW	Minor facts
	NORMAL	Standard conversation summaries
	HIGH	Important information (preferences, key facts)
	CRITICAL	Essential data (passwords, configs)
MemoryTrace	id	Unique identifier
	content	Memory text content
	memory_type	Type (Episodic/Semantic/Procedural/Spatial)
	importance	Importance level
	timestamp	Creation timestamp
	embedding	Vector embedding for similarity search
	metadata	Arbitrary key-value metadata
	room_id	Assigned Memory Palace room
	access_count	Retrieval frequency counter
	last_accessed	Last access timestamp
	decay_factor	Forgetting curve multiplier
	associations	Related memory IDs
MemoryPalaceRoom	id	Room identifier
	name	Room name (e.g. Library, Kitchen)
	description	Room purpose description
	topic	Domain topic
	emotional_valence	Emotional tone of the room
	embedding	Room theme embedding
	created_at	Creation timestamp
	memory_ids	List of contained memory traces
	adjacent_rooms	Related room connections

Table 7.2: Memory Data Models

7.3 Speech Models

2gray!10white

Model	Field	Description
SynthesizeRequest	text	Text to synthesize
	emotion	Emotion style (neutral, happy, excited, serious, calm, angry)
	return_visemes	Whether to return viseme data for lip sync
	humanize	Apply natural speech patterns
	rate	Speech rate multiplier (0.5–2.0)
	pitch	Pitch shift in semitones
	effect	Audio effect (radio, echo, robotic, whisper, slow, fast)
SynthesizeResponse	audio	Base64-encoded audio data
	audio_format	Format (wav, mp3, ogg)
	visemes	Lip sync viseme sequence (optional)
	emotion	Detected/applied emotion
	duration_ms	Audio duration in milliseconds
	cache_hit	Whether result was from cache
	latency_ms	Processing latency
	rate	Applied speech rate
	pitch	Applied pitch shift

Table 7.3: TTS Request/Response Models

Chapter 8

Extended Feature Modules

8.1 Color and Emotion Detection

A comprehensive computer vision subsystem with 12 tools spanning color analysis and face-based emotion recognition. The color pipeline supports named-color mapping (28 named colors via Euclidean distance in RGB space), dominant color extraction via quantized bucket counting, monochromatic palette generation, and remote/local image analysis. The emotion pipeline integrates with DeepFace for face-based emotion/age/gender/race classification, OpenCV Haar Cascade for face detection, and BERT-based text emotion analysis with keyword fallback when transformers are unavailable.

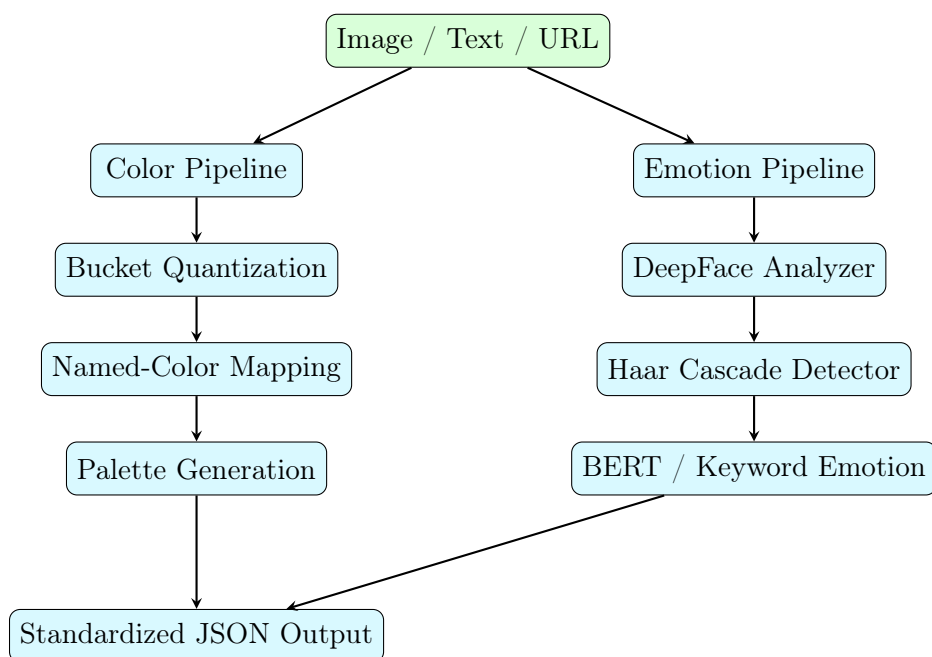


Figure 8.1: Color and Emotion Detection Pipeline

8.2 Shopping Assistant

A full e-commerce assistant module with SQLite-backed product database, FAISS vector search for policy retrieval, and TF-IDF semantic routing. Stores 15 sample products

across 6 categories with real-time search, recommendation scoring (weighted by price, rating, stock, and category match), and order tracking via a simulated state machine.

2gray!10white

Tool	Function
product_search_db	Filtered product search by category, price range, rating
product_get_details	Full product details by ID
product_list_categories	Browsable category tree with product counts
policy_search	Semantic policy lookup via FAISS embeddings
get_product_recommendations	Weighted recommendation scoring
check_store_policy	Return/exchange/shipping policy lookup
track_order	Order status via simulated state machine
shopping_route_query	TF-IDF semantic router (product vs. general)

Table 8.1: Shopping Assistant Tools

8.3 Email and Calendar Agent

A self-contained email management system with SQLite storage, vector-based semantic search, and calendar scheduling. Implements a Human-in-the-Loop (HITL) pattern for sensitive operations.

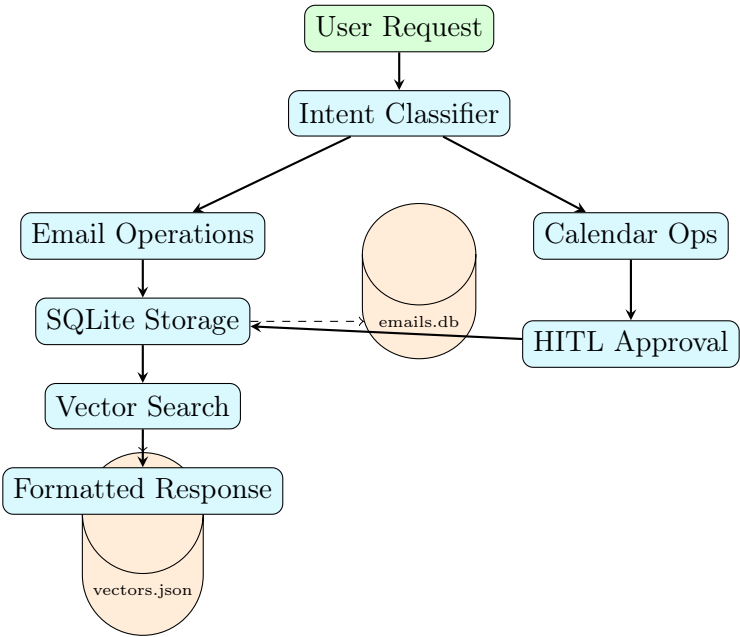


Figure 8.2: Email and Calendar Agent Architecture

Tool
Function
email_search
Semantic + keyword email search over stored messages

Tool
Function
email_list_inbox
List inbox/folder messages with pagination
email_get_message
Full message details (headers, body, attachments)
email_send
Send email with CC support (HITL-gated)
email_draft
Save draft without sending
email_reply
Reply to a message thread
email_mark_read
Mark message as read
email_star
Star/unstar a message
email_archive
Archive a message
email_trash
Move message to trash
email_forward
Forward message with optional note
email_search_by_sender
Filter messages by sender address
email_get_unread_count
Quick unread badge count
email_get_attachment
Download attachment metadata
calendar_list
List events for a date range
calendar_create
Create event with attendees, location, duration (HITL-gated)
calendar_update
Update event fields
calendar_delete
Delete an event (HITL-gated)
calendar_find_slots
Find free time slots across schedule

8.4 Data, Media, and Advanced Tool Libraries

Three expanded tool modules adding 110+ utility functions covering data manipulation, media processing, and advanced text/security operations.

8.4.1 Data Tools (35+ functions)

String operations (reverse, upper, split, join, pad, truncate, replace, regex), list operations (shuffle, chunk, unique, flatten, zip, group, sort), dictionary operations (merge, flatten, pick, omit, sort_keys, rename_key), JSON operations (validate, diff, flatten, depth, merge, filter), math/statistics (mean, median, mode, stddev, correlation, percentile, factorial, gcd, lcm, fibonacci, prime_check).

8.4.2 Media Tools (30+ functions)

Geographic utilities (distance, midpoint, bounding box, DMS conversion, elevation, time-zone, sunrise/sunset), time utilities (now, epoch, diff, add, subtract, weekday, julian day, ISO format, age, leap year), file operations (copy, move, delete, rename, touch, size, modified), HTTP utilities (GET, POST, PUT, DELETE, PATCH with headers, download), format converters (JSON/YAML/XML/CSV/TOML/Markdown interconversion, HTML escape), encoding (base64, URL, slugify).

8.4.3 Advanced Tools (45+ functions)

Text intelligence (summarize, sentiment, keywords, readability, perplexity, plagiarism_check, diff, detect_language), security (hash_text, encrypt/decrypt with Fernet, JWT token generation, password strength, port scan), development tools (uuid, ulid, cron_next, cron_describe, semver_compare, diff_patch), random generators (dice, lottery, tarot, zodiac, haiku, madlib, emoji, morse_code, pig_latin), format/unit converters (bytes, temperature, distance, weight, volume, speed, time, currency, number base, roman numeral).

8.5 Understand-Anything Integration

A complete knowledge graph generation pipeline that converts any codebase into an interactive, explorable architecture visualization. Uses a multi-agent pipeline with tree-sitter deterministic parsing for structural facts (imports, exports, definitions, call sites) and LLM-based semantic analysis for plain-English summaries, architectural layers, business domain mapping, and guided learning tours.

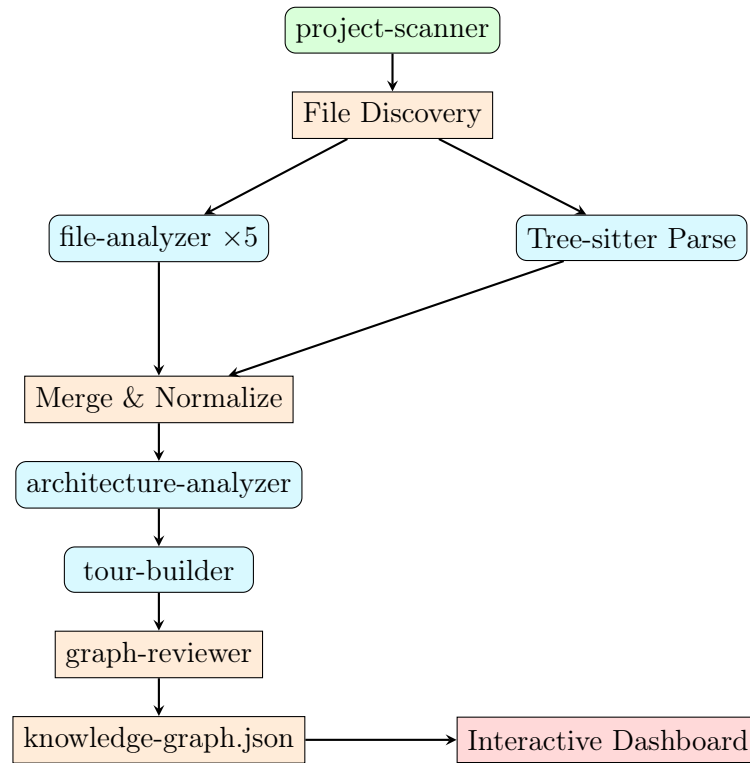


Figure 8.3: Understand-Anything Multi-Agent Pipeline

Capabilities include:

- **/understand** – Multi-agent codebase analysis producing a structured knowledge graph with 13 node types and 26 edge types
- **/understand-dashboard** – Interactive web-based graph viewer with pan/zoom, fuzzy+semantic search, persona-adaptive UI
- **/understand-chat** – Natural language querying over the codebase graph
- **/understand-diff** – Git diff impact analysis showing affected components
- **/understand-explain** – Deep-dive explanations of specific files, functions, or classes
- **/understand-domain** – Business domain flow extraction (domains → processes → steps)
- **/understand-knowledge** – Karpathy-pattern wiki analysis with entity extraction
- **/understand-onboard** – Auto-generated team onboarding guides

Chapter 9

Roadmap

9.1 Feature Categories

The project roadmap (`FEATURE_ROADMAP_200.md`) defines 362 items across 26 categories in 3 priority tiers:

9.1.1 High Priority (Implement First)

1. LLM & AI Improvements (19 items)
2. Memory & Knowledge Base (10 items)
3. Voice & Audio (15 items)
4. Avatar & Expressions (10 items)
5. Vision & Computer Vision (8 items)
6. Smart Home & IoT (18 items)
7. Calendar & Scheduling (12 items)
8. Email & Communication (14 items)
9. Web Search & Info (14 items)
10. Documentation & Knowledge (8 items)

9.1.2 Medium Priority (Implement Next)

11. File & Data Management (16 items)
12. Automation & Workflows (22 items)
13. Development & Coding (18 items)
14. Financial & Analytics (22 items)
15. Health & Wellness (14 items)
16. Entertainment & Gaming (22 items)

- 17. Travel & Automotive (20 items)
- 18. Shopping & E-commerce (10 items)

9.1.3 Low Priority (Nice to Have)

- 19. Learning & Education (12 items)
- 20. Art & Creativity (10 items)
- 21. Home, Garden & Pets (18 items)
- 22. Social & Relationships (10 items)
- 23. AI Agent Capabilities (13 items)
- 24. Augmented Reality (8 items)
- 25. Security & Privacy (12 items)
- 26. Notifications & Alerts (8 items)

Chapter 10

Appendix

10.1 Complete Tool List

The full tool registry spans 300+ tools across 40+ categories. A condensed index is shown below; see the Tool Categories table in Section ?? for the complete breakdown.

Tool Name	Category	Description
get_time	utility	Current date and time
search_web	information	Web search
get_weather	information	Weather forecast
send_email	communication	Send email
create_calendar_event	productivity	Create event
set_reminder	productivity	Set reminder
control_light	smart_home	Smart light control
remember	memory	Store in long-term memory
recall	memory	Retrieve from memory
detect_colors	vision	Dominant colors from camera
read_file	filesystem	Read file contents
write_file	filesystem	Write to file
read_csv	data	Read CSV file
query_json	data	Query JSON by key path
run_python	development	Execute Python sandbox
get_stock_price	finance	Current stock price
convert_currency	finance	Currency conversion
get_joke	fun	Random joke
plot_line_chart	visualization	Line chart
extract_dominant_colors	vision	Dominant colors from image
color_name_from_rgb	vision	Nearest named color
color_palette_generate	vision	Monochromatic palette
detect_emotion_face	vision	Face emotion (DeepFace)
detect_emotion_text	vision	Text emotion (BERT)
detect_faces_image	vision	Face detection (Haar)
analyze_face_attributes	vision	Age/gender/race/emotion
compare_faces	vision	Face similarity
analyze_emotion_realtime	vision	Real-time emotion analysis

Tool Name	Category	Description
product_search_db	shopping	Product database search
policy_search	shopping	Semantic policy lookup
track_order	shopping	Order status
email_search	email	Semantic email search
email_send	email	Send email
calendar_create	calendar	Create event
str_reverse	data	String reversal
list_shuffle	data	Shuffle list
dict_merge	data	Merge dictionaries
stat_mean	data	Statistical mean
geo_distance	media	Geographic distance
time_now	media	Current epoch time
file_copy	media	Copy file
http_get	media	HTTP GET request
json_prettify	media	Pretty-print JSON
hash_text	advanced	SHA-256 hash
encrypt_text	advanced	Fernet encryption
summarize_text	advanced	Text summarization
sentiment_analysis	advanced	Sentiment score
generate_password	security	Strong password
generate_image	creative	DALL-E image generation
notify	system	Desktop notification
... (300+ total across 40+ categories)		

10.2 Requirements

The project depends on a Python 3.12+ stack organized around FastAPI (async web framework), Redis (message broker), ChromaDB (vector database), and multiple AI/ML libraries:

2gray!10white

Category	Dependencies
Web Framework	fastapi, uvicorn, pydantic, aiohttp, websockets, httpx
LLM Clients	openai, anthropic, tiktoken (OpenAI, Claude, DeepSeek, OpenRouter, Ollama, llama.cpp)
Database	chromadb, sentence-transformers, redis[hiredis], sqlite3
Vision	opencv-python, ultralytics (YOLOv8), face-recognition, deepface, pyzbar
Audio	pyaudio, webrtcvad, openai-whisper, TTS (Coqui), gTTS
Data/ML	scikit-learn, pandas, numpy, matplotlib, scipy, transformers, torch
Utilities	pyautogui, paho-mqtt, spotipy, GitPython, apscheduler, python-dotenv, psutil
File Handling	openpyxl, PyMuPDF, python-docx, python-pptx, pyyaml
Security	cryptography, pyjwt, bcrypt
Testing	pytest, pytest-asyncio, pytest-cov

Table 10.2: Python Dependencies by Category

10.3 File Tree

The updated project structure after cleanup:

Directory/File	Description
.env	Environment configuration
.env.example	Environment template
Dockerfile	Backend Docker image
opencode.json	opencode project configuration with skills.paths
README.md	Project overview
docker-compose.yml	Multi-service deployment
equalizer.html	Standalone 3D audio equalizer
requirements.txt	Python dependencies
start_agent.py	CLI entry point
FEATURE_ROADMAP_200.md	362-item roadmap
backend/orchestrator/	FastAPI app, Pydantic models, agent + ToolRegistry
backend/memory/	

Directory/File	Description
	Memory Palace with 20 rooms (ChromaDB)
backend/speech/	Multi-engine TTS, emotion mapping, lip sync
backend/hearing/	PyAudio capture, VAD, Whisper STT
backend/vision/	YOLOv8, face recognition, color/emotion detection
backend/actions/	30+ action modules (APIs, files, web, database, git, email, calendar, smart home, system, scheduler, ...)
backend/tools/	Autoregistered tool libraries: color_emotion_tools, data_tools, media_tools, advanced_tools, shopping_tools, ...
backend/shared/	Config, multi-LLM client, Redis client
frontend/	React 18 + Three.js 3D avatar
docs/	API reference, deployment guide, this .tex doc
scripts/	CLI tool
locale/	Internationalization
.understand-anything/	Understand-Anything scan results, batches, .understandignore